brain/astar.py — Full Line-by-Line Walkthrough

Length: 73 lines. **Role:** classic A* pathfinding over a 25×25 grid map of the arena, used as an alternative/backup route-planner to the Physarum slime-mold planner (`carl_physarum.py`) and the hippocampal replay-navigation system (`carl_grid_cells.py`). This file is not imported anywhere inside `carl_simulation.py` — it's a standalone planning utility that belongs to a different, more classical "cognitive map" (`CM`) representation than the one the live simulation actually uses.

Lines 1–2: Imports

```python
import numpy as np
import heapq
```

`numpy` handles the coordinate/clipping math. `heapq` is Python's built-in binary-heap module — this is what turns a plain list into a priority queue, which is the data structure A* needs to always expand the "most promising" unvisited cell next.

Lines 4–10: Function signature and grid constants

```python
def astar_path(CM, start_pos, goal_pos):
    MAP_RES = 25
    MAP_XMIN = -0.5
    MAP_XMAX = 7.7
    MAP_YMIN = -0.5
    MAP_YMAX = 6.5
```

`CM` ("Cognitive Map") is expected to be a `25×25×N` array — a grid over a rectangular world region, where the first channel (`CM[:,:,0]`) encodes danger/wall likelihood at each cell. `start_pos`/`goal_pos` are continuous, real-world (x, y) coordinates. `MAP_RES=25` — a 25×25 grid. The bounding box (`-0.5..7.7` in x, `-0.5..6.5` in y) is worth noting: this is a *different* coordinate system than `carl_simulation.py`'s arena, which is centered on the origin and roughly `±4.9`. This confirms `astar.py` belongs to an earlier or parallel map convention (likely the older, non-origin-centered arena layout), not the current live world.

Lines 12–15: `_map_cell()` — world coordinates → grid indices

```python
```

```
def _map_cell(xw, yw):
    ix = int(np.clip((xw - MAP_XMIN)/(MAP_XMAX - MAP_XMIN)*MAP_RES, 0, MAP_RE
    iy = int(np.clip((yw - MAP_YMIN)/(MAP_YMAX - MAP_YMIN)*MAP_RES, 0, MAP_RE
    return ix, iy
```

Standard linear rescale: subtract the minimum, divide by the span to get a 0–1 fraction, multiply by the resolution to land on a grid index, then `np.clip` guarantees the result never falls outside `[0, 24]` even if the real-world point is slightly outside the mapped rectangle (e.g. a robot that's briefly out of bounds). This is a closure — it captures `MAP_XMIN` etc. from the enclosing function rather than taking them as parameters, a minor style choice that keeps the call sites terse.

Lines 17–18: Convert start/goal into grid cells

python

```
sx, sy = _map_cell(start_pos[0], start_pos[1])
gx, gy = _map_cell(goal_pos[0], goal_pos[1])
```

Both endpoints get snapped onto the same 25×25 grid the search will run over.

Lines 20–25: Build the binary wall grid (and an unused safety check)

python

```
grid = (CM[:, :, 0] > 0.85).astype(int)
if grid[gx, gy] == 1:
    pass
```

Any cell whose danger channel exceeds `0.85` is treated as an absolute wall for this thresholded view (`grid`). Note: `grid` itself is actually never used again below — the real cost function (line 61) reads directly from `CM[:,:,0]` as a continuous penalty rather than this binarized array, so `grid` and the `if grid[gx,gy]==1: pass` check are dead code — the comment ("Goal is in a wall — shouldn't happen") documents an intended defensive check that was never actually wired up to do anything (a `pass` is a no-op).

Lines 27–35: Initialize the open set and score tables

python

```
open_set = []
heapq.heappush(open_set, (0, sx, sy))
came_from = {}

g_score = {(x,y): float('inf') for x in range(MAP_RES) for y in range(MAP_RES
g_score[(sx, sy)] = 0

f_score = {(x,y): float('inf') for x in range(MAP_RES) for y in range(MAP_RES
f_score[(sx, sy)] = abs(sx - gx) + abs(sy - gy)
```

This is textbook A*. `g_score[cell]` = cheapest known cost to reach that cell from the start; every cell starts at infinity except the start cell itself (cost 0). `f_score[cell] = g_score[cell] + heuristic(cell, goal)` — the heuristic here is Manhattan distance (`|dx| + |dy|`), which is admissible (never overestimates) for a grid that allows 4-directional movement, though the code below actually also allows diagonal moves (see line 57) — for diagonal movement a strictly admissible heuristic would normally be Chebyshev or octile distance, so Manhattan here is a slight mismatch that would make the search marginally less efficient (it can very occasionally expand a few more nodes than optimal) without making the final path wrong, since A* still explores enough of the grid to find the true minimum-cost path. `open_set` is a min-heap of `(priority, x, y)` tuples — heapq compares tuples element-by-element, so it naturally sorts by `f_score` first.

Lines 37–38: Main loop — pop the best candidate

python

```
while open_set:
    _, cx, cy = heapq.heappop(open_set)
```

Pulls the cell with the lowest `f_score` off the heap every iteration — the "always expand the most promising node" rule that makes A* efficient rather than exhaustively brute-force like plain Dijkstra.

Lines 40–55: Goal reached — reconstruct and rescale the path

python

```python
if cx == gx and cy == gy:
    path = []
    curr = (cx, cy)
    while curr in came_from:
        path.append(curr)
        curr = came_from[curr]
    path.reverse()

    cont_path = []
    for (px, py) in path:
        c_x = MAP_XMIN + (px + 0.5) / MAP_RES * (MAP_XMAX - MAP_XMIN)
        c_y = MAP_YMIN + (py + 0.5) / MAP_RES * (MAP_YMAX - MAP_YMIN)
        cont_path.append([c_x, c_y])
    return cont_path
```

Once the popped cell *is* the goal, walk backward through the `came_from` dictionary (which records, for every visited cell, which cell it was reached from) until hitting a cell with no recorded predecessor — that's the start. `.reverse()` flips it back into start→goal order. The rescale step is the inverse of `_map_cell()`: `(index + 0.5)` puts the returned point at the *center* of each grid cell (not its corner), then the same linear formula maps it back into real-world coordinates. Note the start cell itself is never appended to `path` (the loop stops once `curr` has no entry in `came_from`, and the start cell is exactly that "no predecessor" case) — so the returned path is every step *after* the start, ending at the goal, which is the conventional way to hand a path to a controller that already knows its own current position.

Lines 57–70: Expand neighbors (8-directional, distance + danger cost)

python

```python
for dx, dy in [(0,1), (1,0), (0,-1), (-1,0), (1,1), (-1,1), (1,-1), (-1,-1)]:
    nx, ny = cx + dx, cy + dy
    if 0 <= nx < MAP_RES and 0 <= ny < MAP_RES:
        danger_cost = CM[nx, ny, 0] * 50.0
        step_cost = (1.414 if dx != 0 and dy != 0 else 1.0)
        tentative_g = g_score[(cx, cy)] + step_cost + danger_cost

        if tentative_g < g_score[(nx, ny)]:
            came_from[(nx, ny)] = (cx, cy)
            g_score[(nx, ny)] = tentative_g
            f = tentative_g + abs(nx - gx) + abs(ny - gy)
            f_score[(nx, ny)] = f
            heapq.heappush(open_set, (f, nx, ny))
```

Eight neighbor offsets cover the four cardinal directions plus the four diagonals. The bounds check discards any neighbor that would fall off the 25×25 grid. `danger_cost = CM[nx,ny,0] * 50.0` is the key design decision in this file, called out in the inline comment "No absolute walls — everything is just a cost": rather than treating high-danger cells as impassable (which is what the unused `grid` array from line 21 would have done), this A* treats *every* cell as traversable but expensive in proportion to its danger value — a cell with danger 1.0 costs 50 extra "distance units" to enter, heavily discouraging the path from crossing it without making it strictly forbidden (useful if, say, the only route to the goal genuinely requires grazing a risky cell). `step_cost` distinguishes straight moves (cost 1.0) from diagonal moves (cost `√2 ≈ 1.414`) — correct Euclidean distance for a grid, preventing the search from thinking a diagonal hop is "as cheap as" an orthogonal one. The relaxation step (`if tentative_g < g_score[...]`) is the standard A* edge-relaxation: only update a neighbor's recorded cost, predecessor, and heap entry if this new path to it is actually cheaper than any previously found path.

Line 72: No path found

```python
return []
```

If the `while open_set` loop empties out without ever popping the goal cell, there's genuinely no route through the grid — the function returns an empty list rather than raising an exception, letting a calling controller check `if not path:` and fall back to some other behavior.

Summary

This is a clean, correct, if slightly generic, textbook A* implementation with one real design choice worth remembering: it never treats any cell as an absolute wall (despite building an unused wall mask that suggests an earlier version might have). Instead it converts a continuous danger field into a smoothly-scaled traversal cost, so the planner will cross a dangerous region only when there's no meaningfully cheaper way around it. Because it isn't imported by `carl_simulation.py`, it currently functions as a self-contained, ready-to-use module rather than something wired into CARL's live 30 Hz control loop — the live loop instead relies on the hippocampal replay-navigation system and, where present, `carl_physarum.py`'s harmonic-pressure planner.